

Backend Programming With NodeJS and ExpressJS

NodeJS

NodeJS

NodeJS:

- A JavaScript runtime written in C++.
- Can interpret and execute JavaScript.
- Includes support for the NodeJS API.

NodeJS API:

- A set of JavaScript libraries that are useful for creating server programs.

V8 (from Chrome):

- The JavaScript interpreter ("engine") that NodeJS uses to interpret, compile, and execute JavaScript code

NodeJS

NodeJS:

- A JavaScript runtime written in C++.
- Can interpret and execute JavaScript.
- Includes support for the NodeJS API.

Q: What does this mean?

NodeJS API:

- A set of JavaScript libraries that are useful for creating server programs.

V8 (from Chrome):

- The JavaScript interpreter ("engine") that NodeJS uses to interpret, compile, and execute JavaScript code

First: Chrome



Chrome:

- A browser written in C++.
- Can interpret and execute JavaScript code.
- Includes support for the DOM APIs.

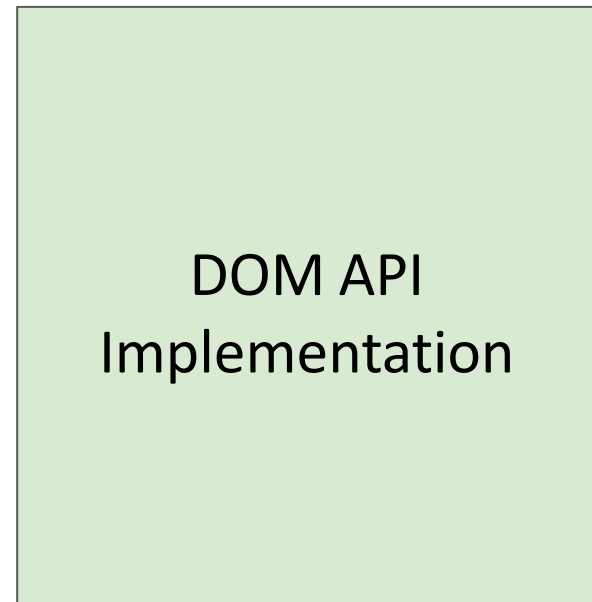
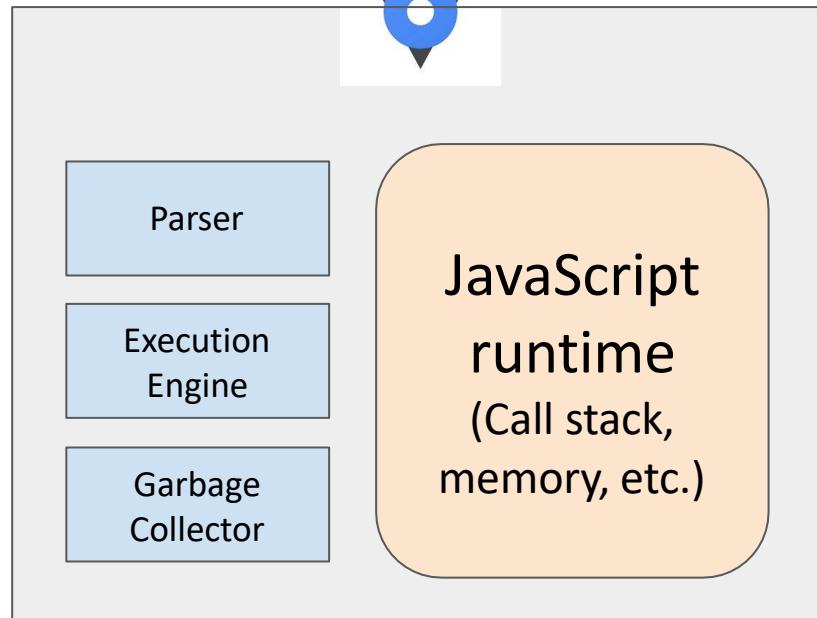
DOM APIs:

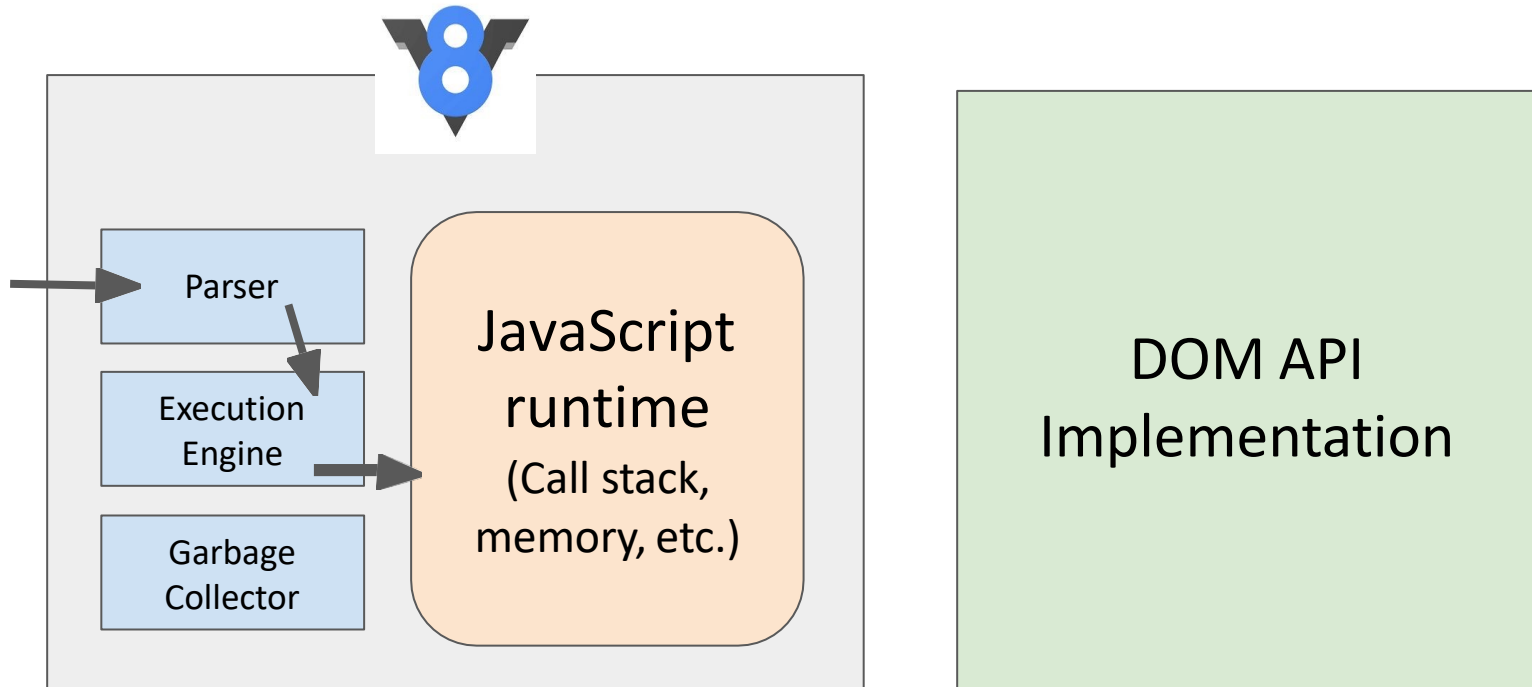
- JavaScript libraries to interact with a web page

V8:

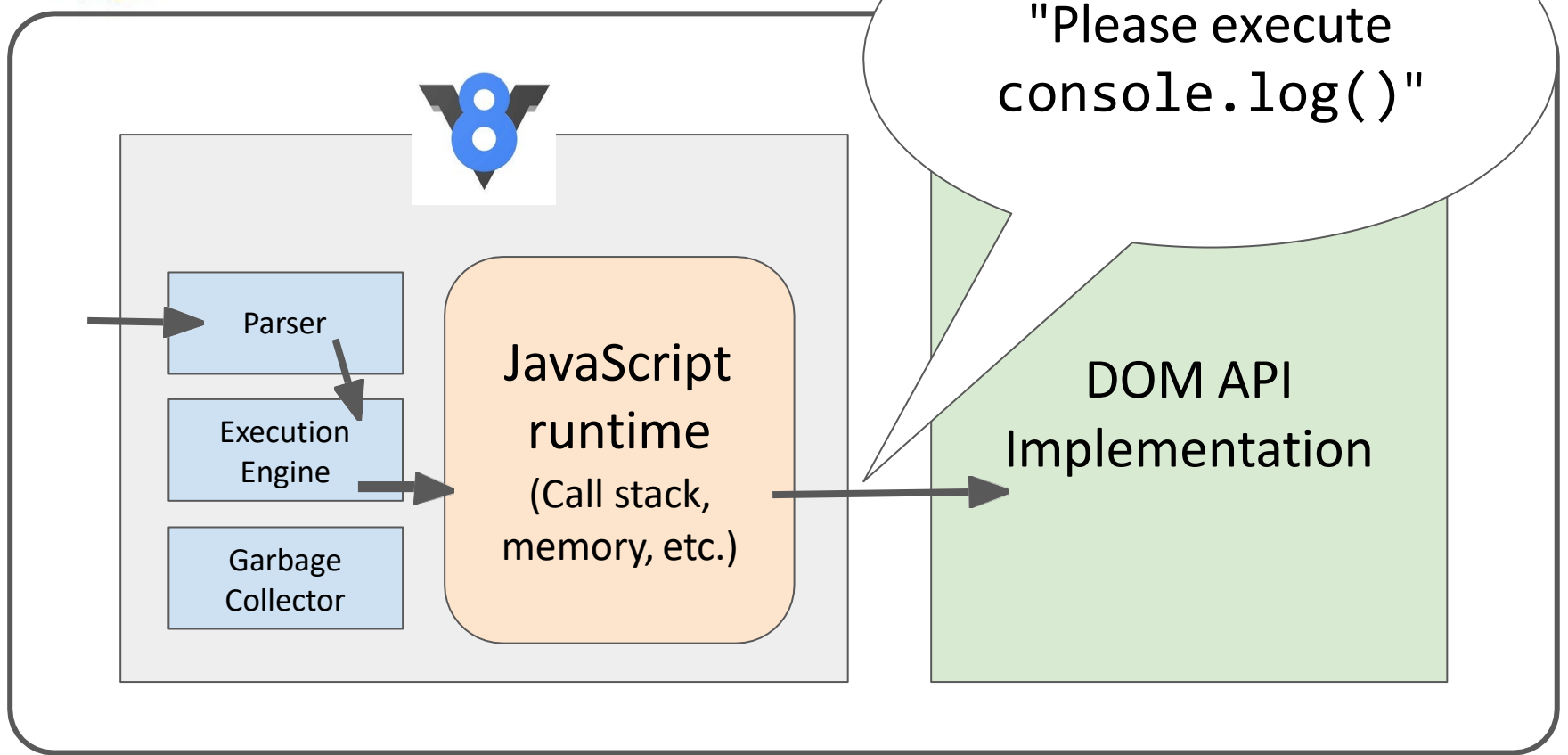
- The JavaScript interpreter ("engine") that Chrome uses to interpret, compile, and execute JavaScript code

Chrome, V8, DOM



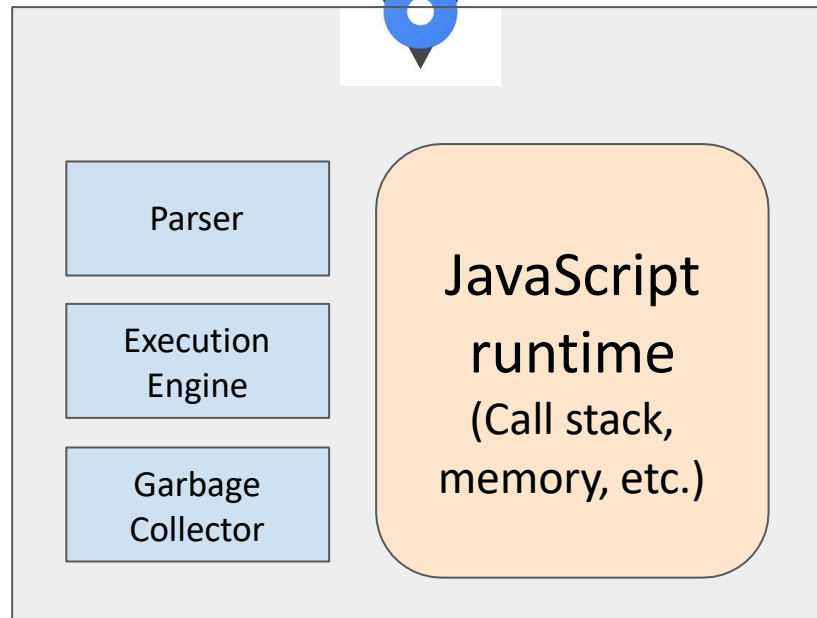


```
const name = 'V8';
```

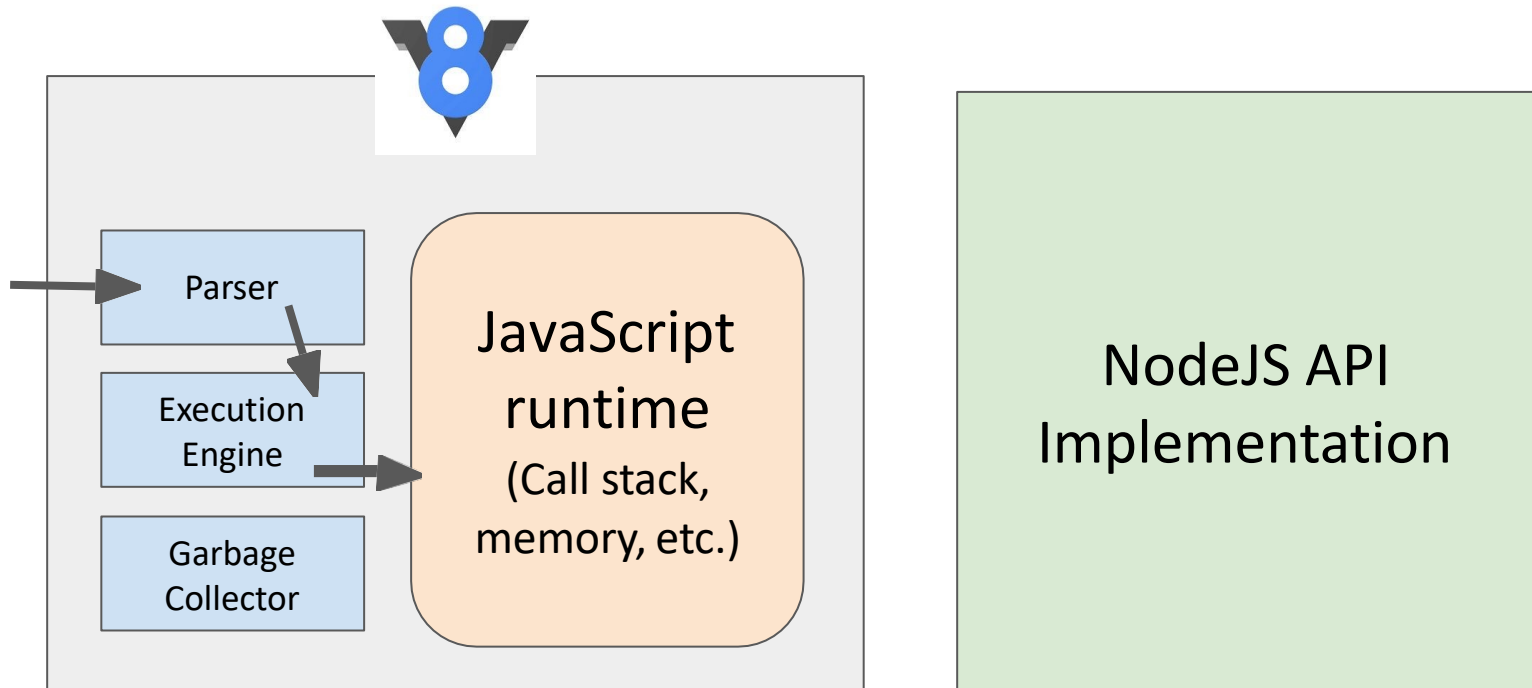


```
console.log('V8');
```

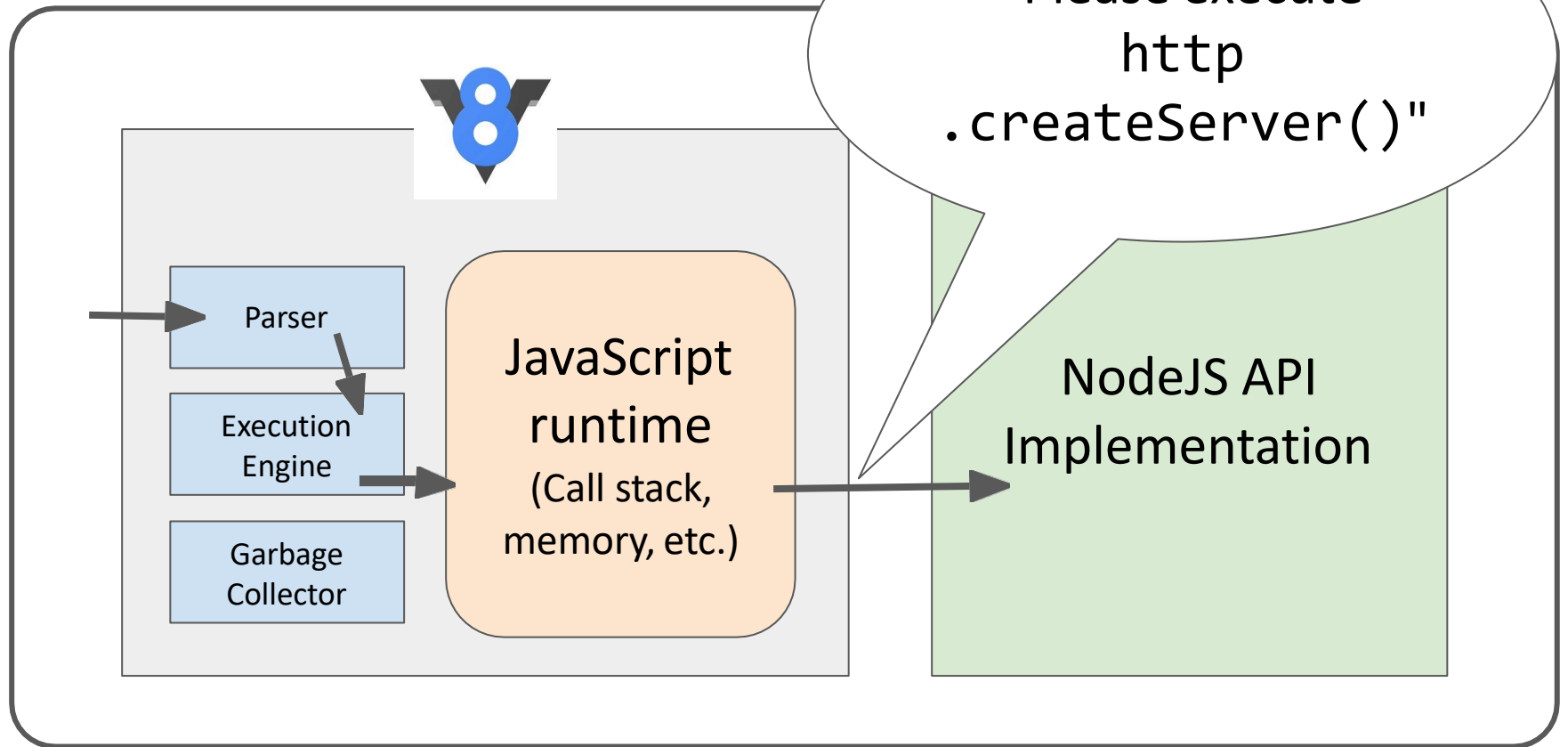
NodeJS, V8, NodeJS APIs



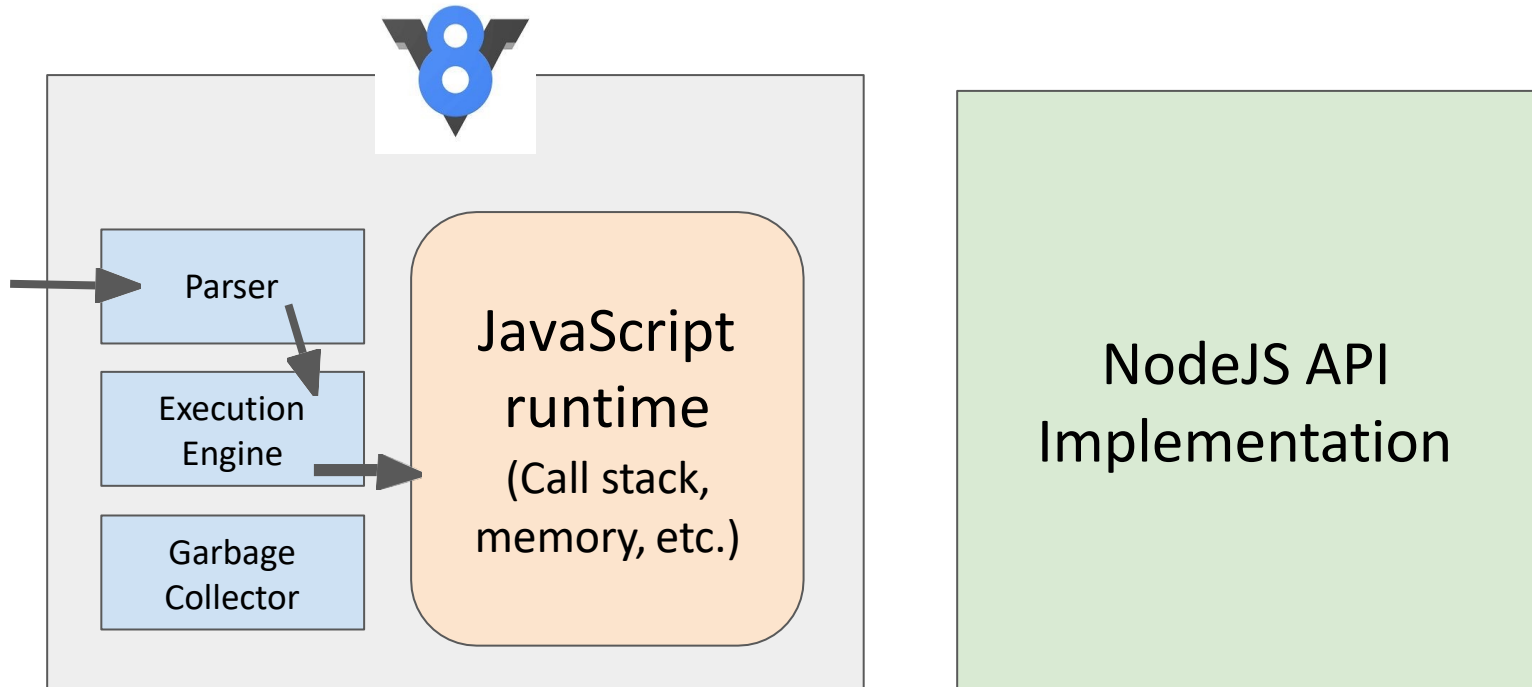
**NodeJS API
Implementation**



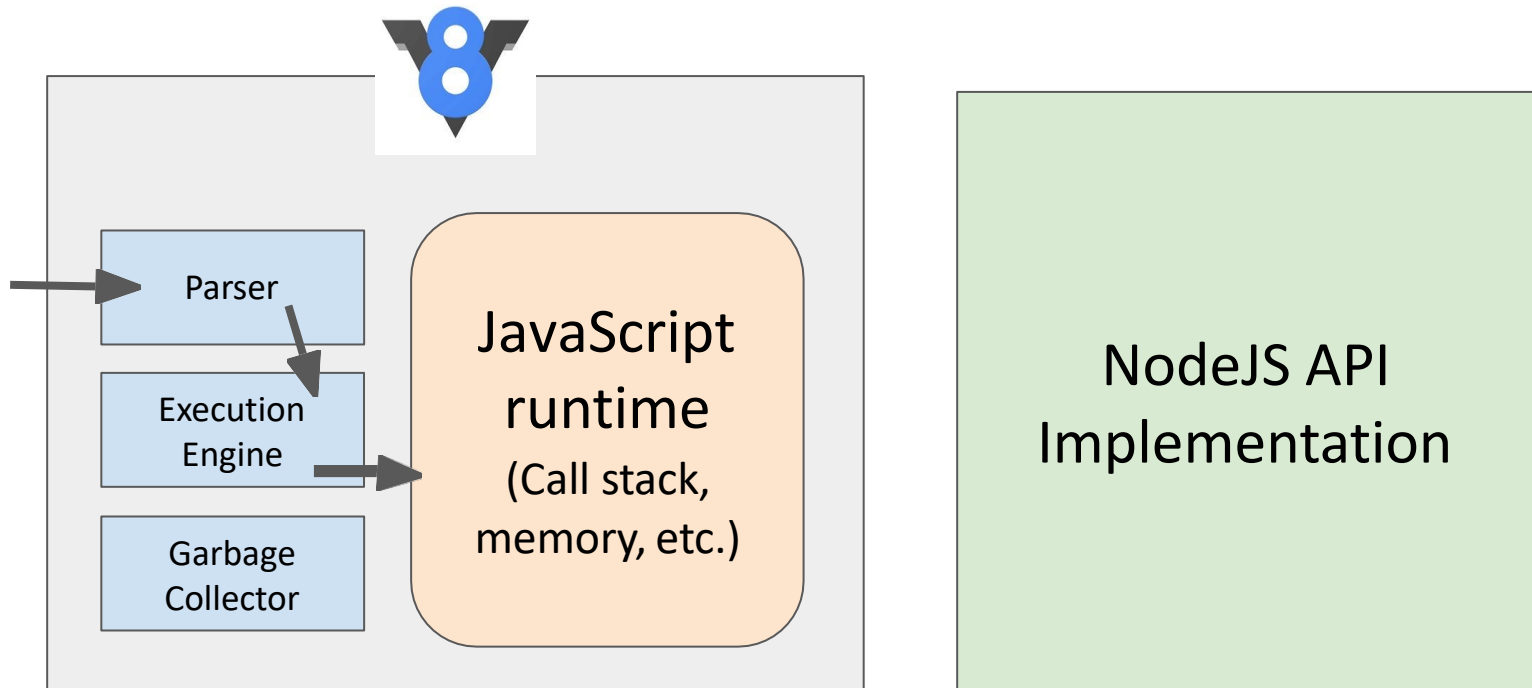
```
const x = 15;  
x++;
```



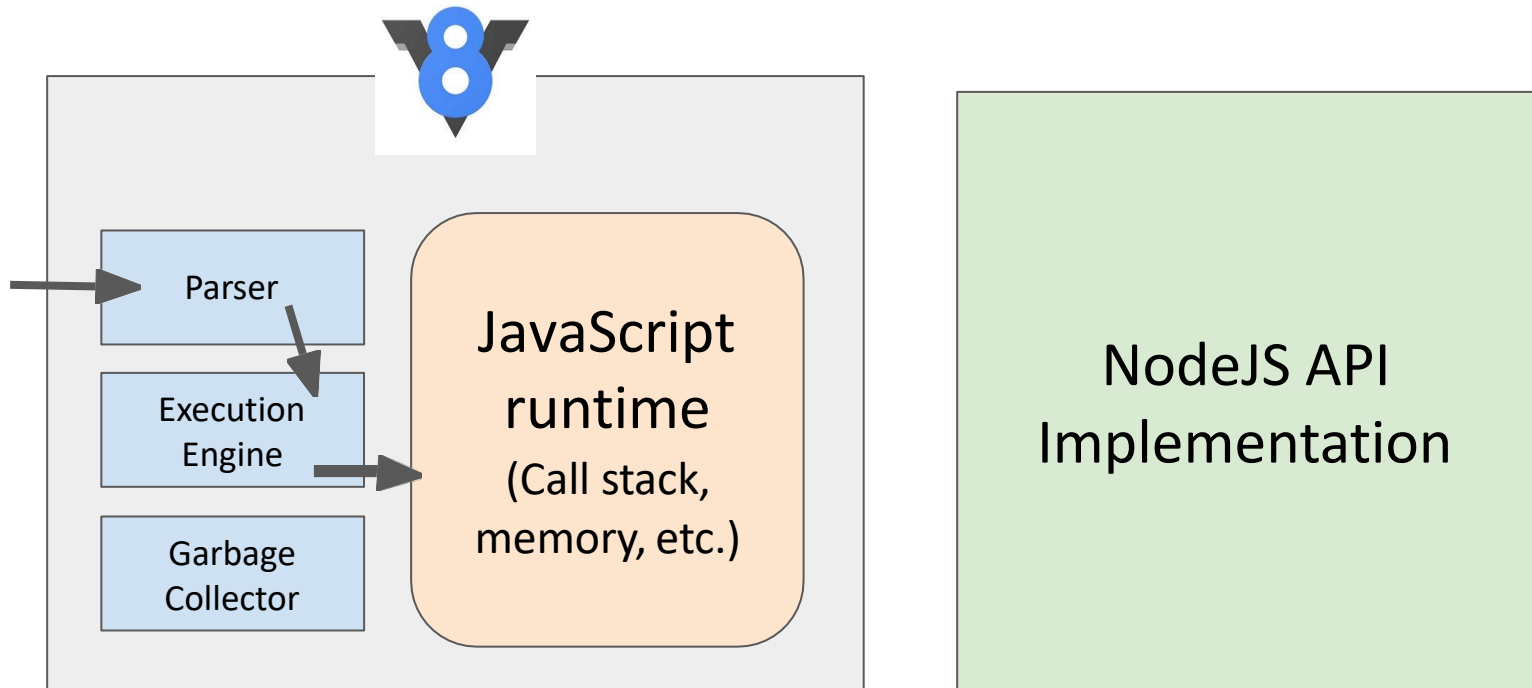
```
http.createServer();
```



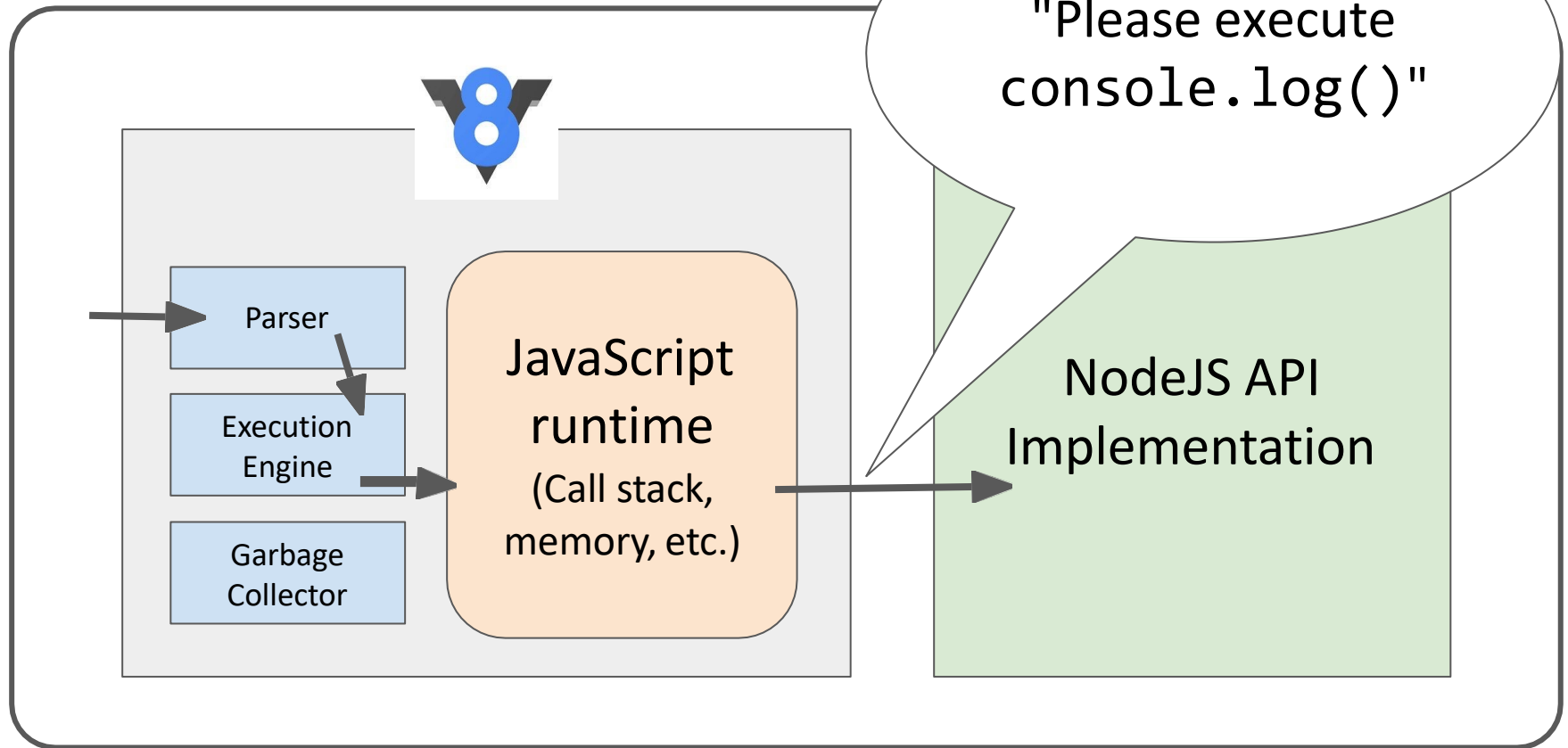
What if you tried to call
`document.querySelector('div')`
; in the NodeJS runtime?



`document.querySelector('div');`
ReferenceError: document is not defined



What if you tried to call `console.log('nodejs');` in the NodeJS runtime?



```
console.log('nodejs');
```

(NodeJS API [implemented their own console.log](#))

NodeJS

NodeJS:

- A JavaScript runtime written in C++.
- Can interpret and execute JavaScript.
- Includes support for the NodeJS API.

NodeJS API:

- A set of JavaScript libraries that are useful for creating server programs.

V8 (from Chrome):

- The JavaScript interpreter ("engine") that NodeJS uses to interpret, compile, and execute JavaScript code

node command

Running node without a filename runs a REPL loop

- Similar to the JavaScript console in Chrome, or when you run "python"

```
$ node
```

```
> let x = 5;
```

```
undefined
```

```
> x++
```

```
5
```

```
> x
```

```
6
```

NodeJS

NodeJS can be used for writing scripts in JavaScript, completely unrelated to servers.

`simple-script.js`

```
function printPoem() {  
  console.log('Roses are red,');  
  console.log('Violets are blue,');  
  console.log('Sugar is sweet,');  
  console.log('And so are you.');
```



```
  console.log();  
}  
  
printPoem();  
printPoem();
```

node command

The node command can be used to execute a JS file:

```
$ node fileName
```

```
$ node simple-script.js
```

```
Roses are red,  
Violets are blue,  
Sugar is sweet,  
And so are you.
```

```
Roses are red,  
Violets are blue,  
Sugar is sweet,  
And so are you.
```

npm

An acronym for Node Package Manager, refers to the Command line utility to install Node.js packages

- Install package locally:

```
$ npm install <module_name>
```

- Use `--save` at the end of the install command to add dependency entry into `package.json` of your application
- The `package.json` file is a JSON file that is used to manage dependencies in Node.js projects.

npm

- The packages.json file can be created by running the npm init command.

```
$ npm init
```

- If all the project dependencies are placed in the package.json file, all of them will be installed at once by simply running npm install

```
$ npm install
```

- Can update or uninstall a package (see more on NPM website)

Node for servers

Here is a very basic server written for NodeJS:

```
const http = require('http');

const server = http.createServer();

server.on('request', function(req, res) {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');
});

server.on('listening', function() {
  console.log('Server running!');
});

server.listen(3000);
```

(WARNING: We will **not actually be writing servers like this!!!**

We will be using ExpressJS to help, but we haven't gotten there yet.

require()

```
const http = require('http');  
const server = http.createServer();
```

The NodeJS `require()` statement loads a module, similar to `import` in Java or `include` in C++.

- We can `require()` modules included with NodeJS, or modules we've written ourselves.
- In this example, 'http' is referring to the [HTTP NodeJS module](#)

require()

```
const http = require('http');  
  
const server = http.createServer();
```

The `http` variable returned by `require('http')` can be used to make calls to the HTTP API:

- `http.createServer\(\)` creates a `Server` object

Emitter.on

```
server.on('request', function(req, res) {  
  res.statusCode = 200;  
  res.setHeader('Content-Type', 'text/plain');  
  res.end('Hello World\n');  
});
```

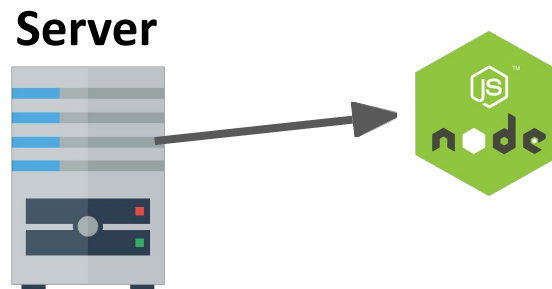
```
server.on('listening', function() {  
  console.log('Server running!');  
});
```

The [on\(\)](#) function is the NodeJS equivalent of `addListener`.

Emitter.on

```
server.on('request', function(req, res) {  
  res.statusCode = 200;  
  res.setHeader('Content-Type', 'text/plain');  
  res.end('Hello World\n');  
});
```

The [request](#) event is emitted each time there is a new HTTP request for the NodeJS program to process.



Emitter.on

```
server.on('request', function(req, res) {  
  res.statusCode = 200;  
  res.setHeader('Content-Type', 'text/plain');  
  res.end('Hello World\n');  
});
```

The [req](#) parameter gives information about the incoming request, and the [res](#) parameter is the response parameter that we write to via method calls.

- [statusCode](#): Sets the HTTP status code.
- [setHeader\(\)](#): Sets the HTTP headers.
- [end\(\)](#): Writes the message to the response body then signals to the server that the message is complete.

listen() and listening

```
server.on('listening', function() {  
  console.log('Server running!');  
});
```

```
server.listen(3000);
```

The listen() function will start accepting connections on the given **port number**.

- The listening event will be emitted when the server has been bound to a port.

Q: What's a port? What is binding?

Development server

```
server.on('listening', function() {  
  console.log('Server running!');  
});
```

```
server.listen(3000);
```

For our development server, we can choose whatever port number we want. In this example, we've chosen 3000.

Running the server

When we run `node server.js` in the terminal, we see the following:

```
vrk:node-server $ node server.js  
Server running!
```

The process does not end after we run the command, as it is now waiting for HTTP requests on port 3000.

Q: How do we send an HTTP request on port 3000?

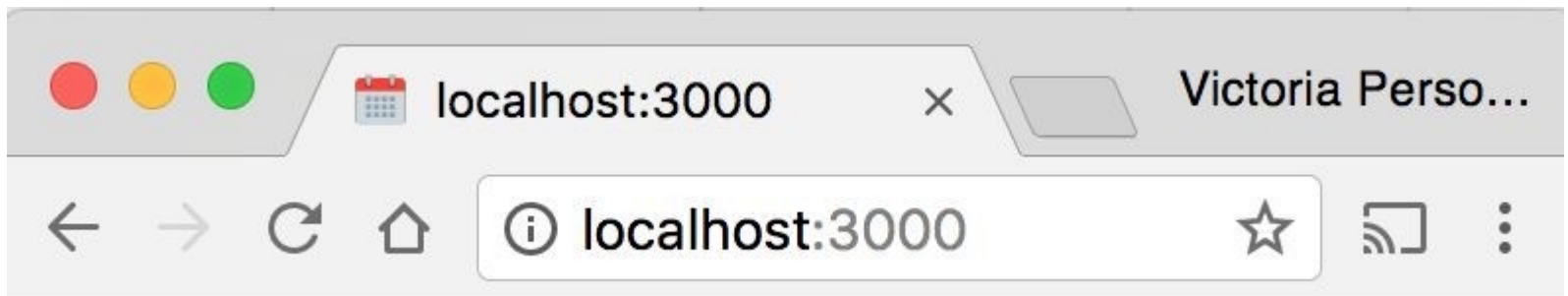
Localhost

We can send an HTTP GET request running on one of the ports on the local computer using the URL:

`http://localhost:portNumber`, e.g.

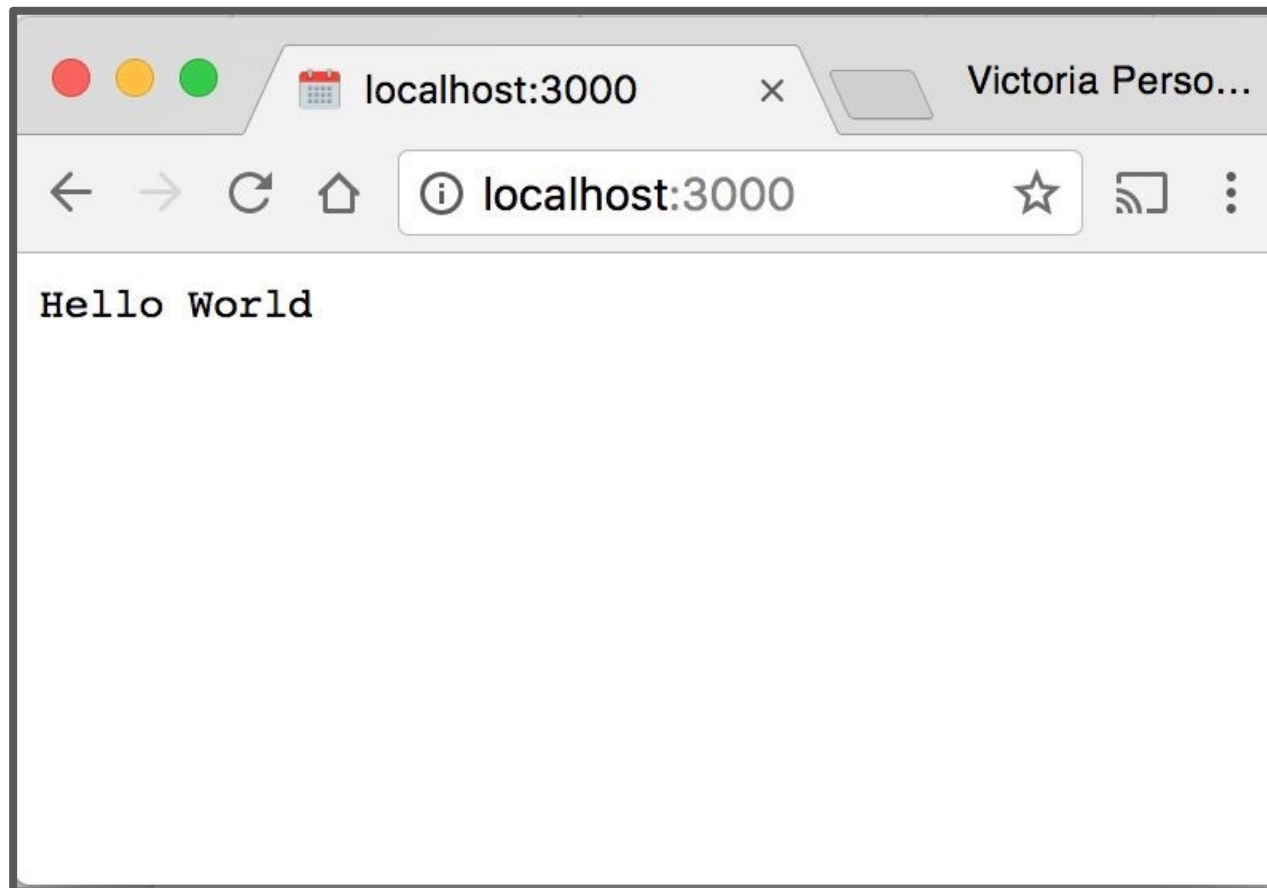
<http://localhost:3000>

[Localhost](#) is a hostname that means "this computer."



Server response

Here is the result of the request to our HTTP server:



Node for servers

This server
returns the same
response no
matter what the
request is.

```
const http = require('http');

const server = http.createServer();

server.on('request', function(req, res) {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');
});

server.on('listening', function() {
  console.log('Server running!');
});

server.listen(3000);
```

Node for servers

The NodeJS server APIs are actually pretty low-level:

- You build the request manually
- You write the response manually
- There's a lot of tedious processing code

```
var http = require('http');

http.createServer(function(request, response) {
  var headers = request.headers;
  var method = request.method;
  var url = request.url;
  var body = [];
  request.on('error', function(err) {
    console.error(err);
  }).on('data', function(chunk) {
    body.push(chunk);
  }).on('end', function() {
    body = Buffer.concat(body).toString();
    // BEGINNING OF NEW STUFF

    response.on('error', function(err) {
      console.error(err);
    });

    response.statusCode = 200;
    response.setHeader('Content-Type', 'application/json');
    // Note: the 2 lines above could be replaced with this next one:
    // response.writeHead(200, {'Content-Type': 'application/json'})

    var responseBody = {
      headers: headers,
      method: method,
      url: url,
      body: body
    };
  });
```

ExpressJS

We're going to use a library called ExpressJS on top of NodeJS:

```
const express = require('express');
const app = express();

app.get('/', function (req, res) {
  res.send('Hello World!');
})

app.listen(3000, function () {
  console.log('Example app listening on port 3000!');
})
```

Express routing

ExpressJS

However, Express is not part of the NodeJS APIs.
If we try to use it like this, we'll get an error:

```
const express = require('express');  
const app = express();
```

```
module.js:327  
  throw err;  
  ^  
  
Error: Cannot find module 'express'  
    at Function.Module._resolveFilename (module.js:327:11)  
    at Function.Module._load (module.js:286:23)  
    at Module.require (module.js:309:17)  
    at require (internal/module.js:20:28)
```

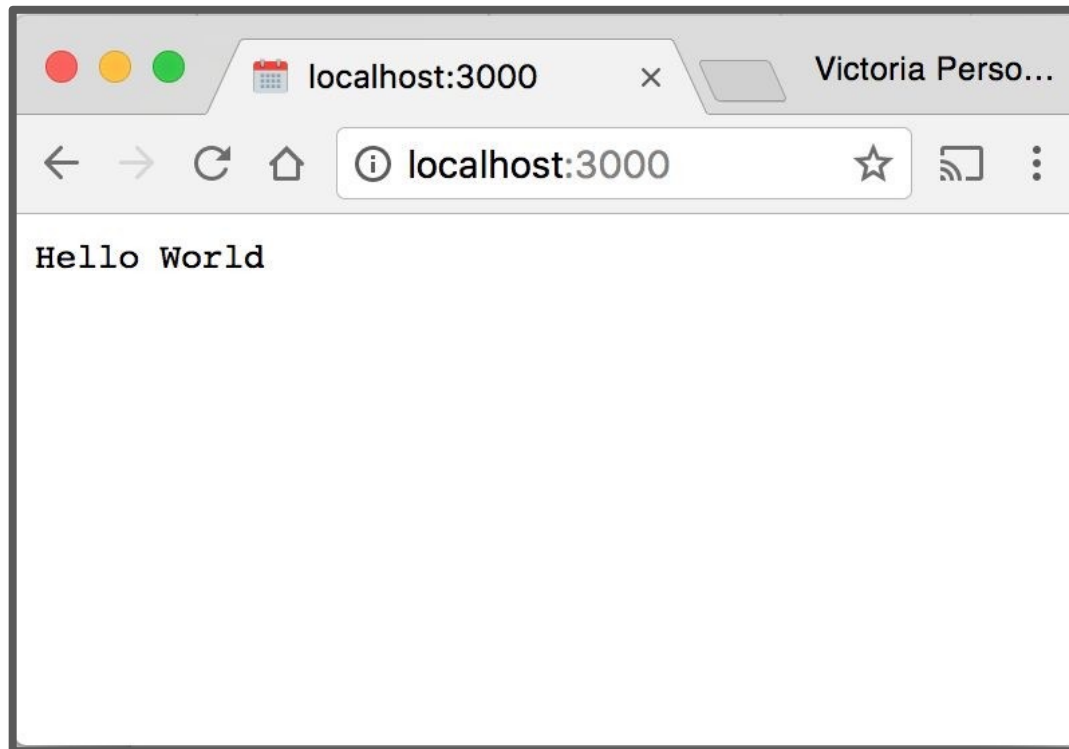
We need to install Express via npm.

Express example

```
$ npm install express
```

```
$ node server.js
```

Example app listening on port 3000!



Express routes

You can specify [routes in Express](#):

```
app.get('/', function (req, res) {  
  res.send('Main page!');  
});
```

```
app.get('/hello', function (req, res) {  
  res.send('GET hello!');  
});
```

```
app.post('/hello', function (req, res) {  
  res.send('POST hello!');  
});
```

Express routes

```
app.get('/hello', function (req, res) {  
  res.send('GET hello!');  
});
```

`app.method(path, handler)`

- Specifies how the server should handle HTTP *method* requests made to URL/*path*
- This example is saying:
 - When there's a GET request to <http://localhost:3000/hello>, respond with the text "GET hello!"

Handler parameters

```
app.get('/hello', function (req, res) {  
  res.send('GET hello!');  
});
```

Express has its own [Request](#) and [Response](#) objects:

- req is a Request object
- res is a Response object
- [res.send\(\)](#) sends an HTTP response with the given content
 - Sends content type "text/html" by default

Querying our server

HTTP requests

Our server is written to respond to HTTP requests ([GitHub](#)):

```
const express = require('express');
const app = express();

app.get('/', function (req, res) {
  res.send('Hello World!');
})

app.listen(3000, function () {
  console.log('Example app listening on port 3000!');
})
```

Q: How do we sent HTTP requests to our server?

Querying our server

Here are three ways to send HTTP requests to our server:

1. Navigate to `http://localhost:3000/<path>` in our browser
 - a. **Caveat:** Can only do GET requests
2. Call `fetch()` in web page
 - a. We've done GET requests so far, but can send any type of HTTP request
3. `curl` command-line tool
 - a. Debug tool we haven't seen yet

curl

curl: Command-line tool to send and receive data from a server ([Manual](#))

`curl --request METHOD url`

e.g.

```
$ curl --request POST http://localhost:3000/hello
```

Querying with fetch()

We can try querying our server the same way we've queried the Spotify or Giphy servers, i.e. via the `fetch()` command:

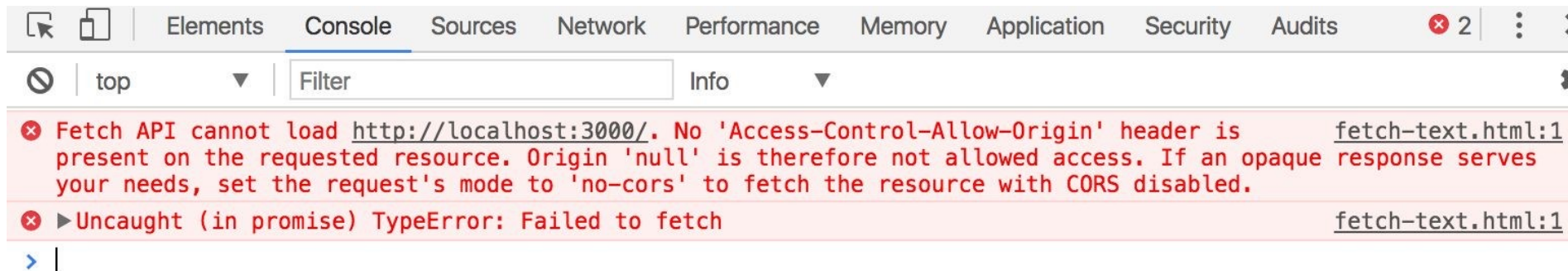
```
function onTextReady(text) {  
    console.log(text);  
}  
  
function onResponse(response) {  
    return response.text();  
}  
  
fetch('http://localhost:3000/')  
    .then(onResponse)  
    .then(onTextReady);
```

fetch() to localhost

But if we try fetching to localhost from different origin:

```
fetch('http://localhost:3000')  
  .then(onResponse)  
  .then(onTextReady);
```

We get this CORS error:



Recall: CORS

CORS: Cross-Origin Resource Sharing ([wiki](#))

- Browser policies for what resources a web page can load
- You **cannot** make cross-origin requests by default for:
 - Resources loaded via `fetch()` or XHR

The problem is that we are trying to `fetch()`

`http://localhost:3000` from a different origin:

- Since the two resources have different origins, this is disallowed by default CORS policy

Enable CORS

```
app.get('/', function (req, res) {  
  res.header("Access-Control-Allow-Origin", "*");  
  res.send('Main page!');  
});
```

You can set an [Access-Control-Allow-Origin](#) HTTP header before sending your response.

- This is the server saying to the browser in its response:
"Hey browser, I'm totally fine with websites of any origin requesting this file."

Enable CORS

Enable *All* CORS Requests

```
const express = require('express')
const cors = require('cors')
const app = express()
app.use(cors())

app.get('/products/:id', function (req, res, next) {
  res.json({msg: 'This is CORS-enabled for all origins!'})
})
```

Configuring CORS

```
const express = require('express')
const cors = require('cors')
const app = express()
const corsOptions = {origin: 'http://example.com'}

app.use(cors(corsOptions))

app.get('/products/:id', function (req, res, next) {
  res.json({msg: 'This is CORS-enabled for some origins!'})
})
```

Different fetch() methods

fetch() with POST

```
app.post('/hello', function (req, res) {  
  res.send('POST hello!');  
});
```

On the server-side, we have defined a function in `app.post()` to handle POST requests to `/hello`.

Q: How do we make a POST request via `fetch()`?

Changing the fetch() method

```
fetch('/hello', { method: 'POST' })  
  .then(onResponse)  
  .then(onTextReady);
```

Q: How do we make a POST request via fetch()?

A: We can change the HTTP method via a second parameter to `fetch()`, which specifies [an options object](#):

- `method`: specifies the HTTP request method, e.g. POST, PUT, PATCH, DELETE, etc.
- GET is the default value.

fetch() with POST

```
function onTextReady(text) {  
  console.log(text);  
}
```

```
function onResponse(response) {  
  return response.text();  
}
```

```
fetch('/hello', { method: 'POST' })  
  .then(onResponse)  
  .then(onTextReady);
```

[GitHub](#)

Sending data to the server

Route parameters

When we used the Spotify API, we saw a few ways to send information to the server via our `fetch()` request.

Example: Spotify Album API

`https://api.spotify.com/v1/albums/7aDBFWp72Pz4NZEtVBANi9`

- The last part of the URL is a **parameter** representing the album id, 7aDBFWp72Pz4NZEtVBANi9

A parameter defined in the URL of the request is often called a "**route parameter**."

Route parameters

Q: How do we read route parameters in our server?

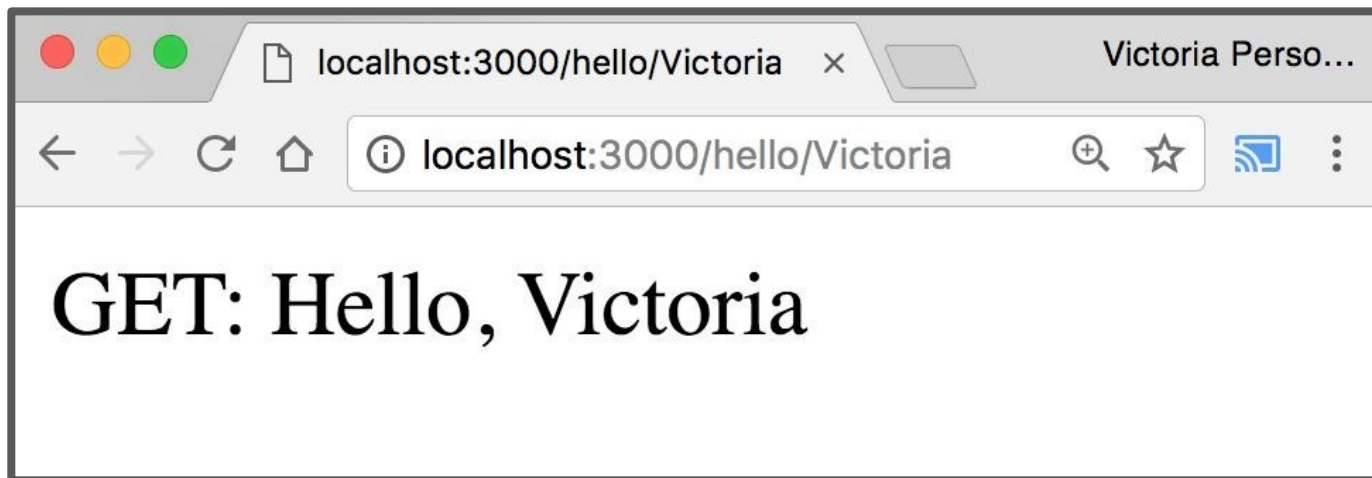
A: We can use the **:*variableName*** syntax in the path to specify a route parameter ([Express docs](#)):

```
app.get('/hello/:name', function (req, res) {  
  const routeParams = req.params;  
  const name = routeParams.name;  
  res.send('GET: Hello, ' + name);  
});
```

We can access the route parameters via **req.params**.

Route parameters

```
app.get('/hello/:name', function (req, res) {  
  const routeParams = req.params;  
  const name = routeParams.name;  
  res.send('GET: Hello, ' + name);  
});
```

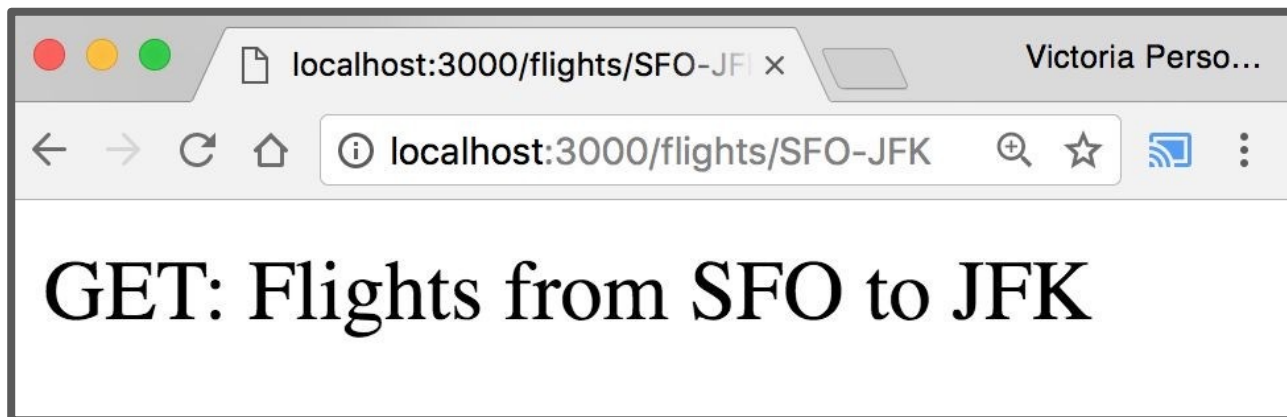


[GitHub](#)

Route parameters

You can define multiple route parameters in a URL ([docs](#)):

```
app.get('/flights/:from-:to', function (req, res) {  
  const routeParams = req.params;  
  const from = routeParams.from;  
  const to = routeParams.to;  
  res.send('GET: Flights from ' + from + ' to ' + to);  
});
```



Query parameters

The Spotify Search API was formed a little differently:

Example: Spotify Search API

`https://api.spotify.com/v1/search?type=album
&q=beyonce`

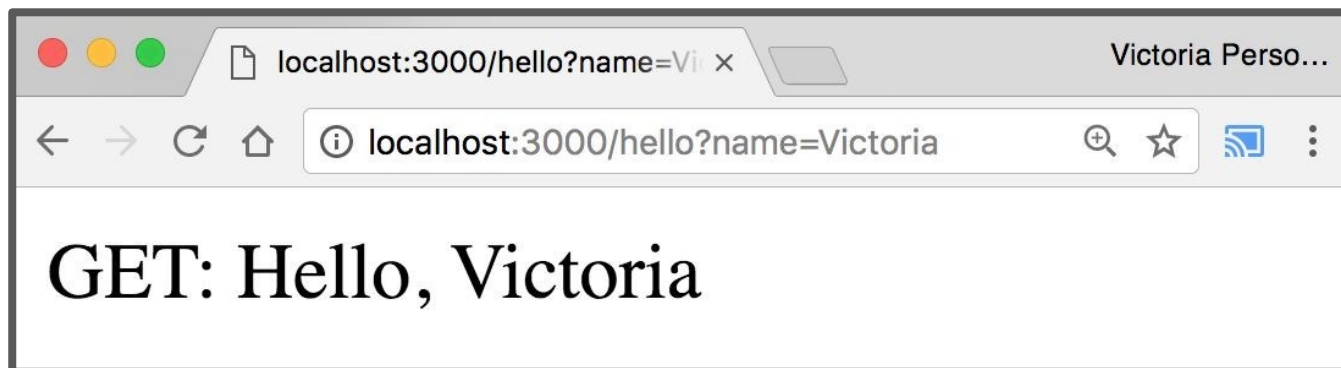
- There were two query parameters sent to the Spotify search endpoint:
 - `type`, whose value is `album`
 - `q`, whose value is `beyonce`

Query parameters

Q: How do we read query parameters in our server?

A: We can access query parameters via **req.query**:

```
app.get('/hello', function (req, res) {  
  const queryParams = req.query;  
  const name = queryParams.name;  
  res.send('GET: Hello, ' + name);  
});
```



Query params with POST

You can send query parameters via POST as well:

```
function onTextReady(text) {  
  console.log(text);  
}  
  
function onResponse(response) {  
  return response.text();  
}  
  
fetch('/hello?name=Victoria', { method: 'POST' })  
  .then(onResponse)  
  .then(onTextReady);
```

(WARNING: We will **not be making POST requests like this!**

We will be sending data in the body of the request instead of via query params.)

Query params with POST

These parameters are accessed the same way:

```
app.post('/hello', function (req, res) {  
  const queryParams = req.query;  
  const name = queryParams.name;  
  res.send('POST: Hello, ' + name);  
});
```

[GitHub](#)

(WARNING: We will **not be making POST requests like this!**

We will be sending data in the body of the request instead of via query params.)

POST message body

However, generally it is poor style to send data via query parameters in a POST request.

Instead, you should specify a message body in your `fetch()` call:

```
const message = {  
  name: 'Victoria',  
  email: 'vrk@stanford.edu'  
};  
const serializedMessage = JSON.stringify(message);  
fetch('/helloemail', { method: 'POST', body: serializedMessage })  
  .then(onResponse)  
  .then(onTextReady);
```

POST message body

Handling the message body in NodeJS/Express is a little messy ([GitHub](#)):

```
app.post('/helloemail', function (req, res) {  
  let data = '';  
  req.setEncoding('utf8');  
  req.on('data', function(chunk) {  
    data += chunk;  
  });  
  
  req.on('end', function() {  
    const body = JSON.parse(data);  
    const name = body.name;  
    const email = body.email;  
    res.send('POST: Name: ' + name + ', email: ' + email);  
  });  
});
```

body-parser

We can use the [body-parser library](#) to help:

```
const bodyParser = require('body-parser');
```

This is not a NodeJS API library, so we need to install it:

```
$ npm install body-parser
```

body-parser

We can use the [body-parser library](#) to help:

```
const bodyParser = require('body-parser');  
const jsonParser = bodyParser.json();
```

This creates a JSON parser stored in `jsonParser`, which we can then pass to routes whose message bodies we want parsed as JSON.

POST message body

Now instead of this code:

```
app.post('/helloemail', function (req, res) {  
  let data = '';  
  req.setEncoding('utf8');  
  req.on('data', function(chunk) {  
    data += chunk;  
  });  
  
  req.on('end', function() {  
    const body = JSON.parse(data);  
    const name = body.name;  
    const email = body.email;  
    res.send('POST: Name: ' + name + ', email: ' + email);  
  });  
});
```

POST message body

We can access the message body through `req.body`:

```
app.post('/helloparsed', jsonParser, function (req, res) {  
  const body = req.body;  
  const name = body.name;  
  const email = body.email;  
  res.send('POST: Name: ' + name + ', email: ' + email);  
});
```

[GitHub](#)

POST message body

We can access the message body through `req.body`:

```
app.post('/helloparsed', jsonParser, function (req, res) {  
  const body = req.body;  
  const name = body.name;  
  const email = body.email;  
  res.send('POST: Name: ' + name + ', email: ' + email);  
});
```

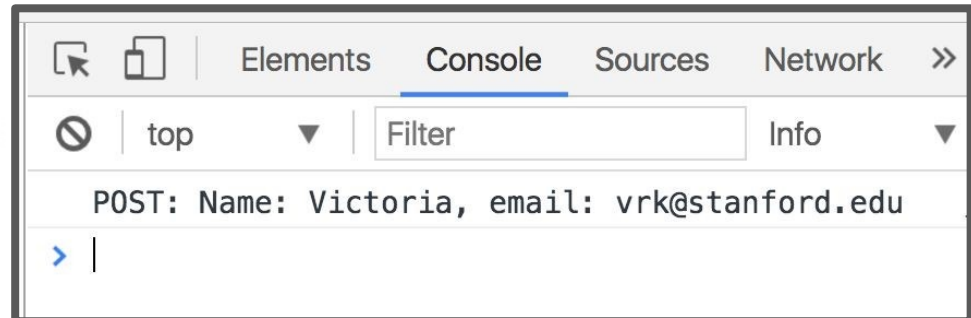
[GitHub](#)

Note that we also had to add the `jsonParser` as a parameter when defining this route.

POST message body

Finally, we need to add JSON content-type headers on the `fetch()`-side ([GitHub](#)):

```
const message = {
  name: 'Victoria',
  email: 'vrk@stanford.edu'
};
const fetchOptions = {
  method: 'POST',
  headers: {
    'Accept': 'application/json',
    'Content-Type': 'application/json'
  },
  body: JSON.stringify(message)
};
fetch('/helloparsed', fetchOptions)
  .then(onResponse)
  .then(onTextReady);
```



Recap

You can deliver parameterized information to the server in the following ways:

1. Route parameters
2. GET request with query parameters
(**DISCOURAGED**: POST with query parameters)
3. POST request with message body

Q: When do you use route parameters vs query parameters vs message body?

GET vs POST

- Use [GET](#) requests for retrieving data, not writing data
 - Use [POST](#) requests for writing data, not retrieving data
- You can also use more specific HTTP methods:
- PATCH: Updates the specified resource
 - DELETE: Deletes the specified resource

There's nothing technically preventing you from breaking these rules, but you should use the HTTP methods for their intended purpose.

Route params vs Query params

Generally follow these rules:

- Use **route parameters** for required parameters for the request
- Use **query parameters** for:
 - Optional parameters
 - Parameters whose values can have spaces

These are conventions and are not technically enforced, nor are they followed by every REST API.

Example: Spotify API

The Spotify API mostly followed these conventions:

<https://api.spotify.com/v1/albums/7aDBFWp72Pz4NZEtVBANi9>

- The Album ID is required and it is a route parameter.

<https://api.spotify.com/v1/search?type=album&q=the%20wee%20knd&limit=10>

- q is required but might have spaces, so it is a query parameter
- limit is optional and is a query parameter
- type is required but is a query parameter (breaks convention)

Notice both searches are GET requests, too

package.json

Installing dependencies

In our examples, we had to install the `express` and `body-parser` npm packages.

```
$ npm install express  
$ npm install body-parser
```

These get written to the `node_modules` directory.

Uploading server code

When you upload NodeJS code to a GitHub repository (or any code repository), **you should not upload the `node_modules` directory**:

- You shouldn't be modifying code in the `node_modules` directory, so there's no reason to have it under version control
- This will also increase your repo size significantly

Q: But if you don't upload the `node_modules` directory to your code repository, how will anyone know what libraries they need to install?

Managing dependencies

If we don't include the `node_modules` directory in our repository, we need to somehow tell other people what `npm` modules they need to install.

`npm` provides a mechanism for this: [package.json](#)

package.json

You can put a file named [package.json](#) in the root directory of your NodeJS project to specify metadata about your project.

Create a [package.json](#) file using the following command:
`$ npm init`

This will ask you a series of questions then generate a `package.json` file based on your answers.

Auto-generated package.json

```
{
  "name": "fetch-to-server",
  "version": "1.0.0",
  "description": "Example of fetching to a server",
  "main": "server.js",
  "dependencies": {
    "body-parser": "^1.17.1",
    "express": "^4.15.2"
  },
  "devDependencies": {},
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "author": "Victoria Kirst",
  "license": "ISC"
}
```

[GitHub](#)

Saving deps to package.json

Now when you install packages, you should pass in the `--save` parameter:

```
$ npm install --save express  
$ npm install --save body-parser
```

This will also add an entry for this library in package.json.

```
"dependencies": {  
  "body-parser": "^1.17.1",  
  "express": "^4.15.2"  
},
```

Saving deps to package.json

If you remove the node_modules directory:

```
$ rm -rf node_modules
```

You can install your project dependencies again via:

```
$ npm install
```

- This also allows people who have downloaded your code from GitHub to install all your dependencies with one command instead of having to install all dependencies individually.

npm scripts

Your package.json file also defines scripts:

```
"scripts": {  
  "test": "echo \"Error: no test specified\" && exit 1",  
  "start": "node server.js"  
},
```

You can run these scripts using `$ npm scriptName`

E.g. the following command runs "node server.js"

```
$ npm start
```

Improve the simple-blog

Change Posts Structure

Backend:

```
const BlogPosts = [  
  {  
    slug: "first-blog-post",  
    title: "First Blog Post",  
    description: "Lorem ipsum dolor sit amet, consectetur  
      adipiscing.",  
  },  
  {  
    slug: "second-blog-post",  
    title: "Second Blog Post",  
    description: "Hello React Router v6",  
  },  
];  
  
module.exports = { BlogPosts };
```

Add New Post

Backend (index.js):

```
const express = require("express");
const app = express();
const cors = require("cors");
const BlogPosts = require("./BlogPosts");
const bodyParser = require("body-parser");
const jsonParser = bodyParser.json();
app.use(cors());

app.post("/api/post", jsonParser, (req, res) => {
  const post = {
    slug: req.body.slug,
    title: req.body.title,
    description: req.body.description,
  };
  BlogPosts.BlogPosts.push(post);
  res.status(200).send({ message: "Posted successful" });
});
```

Add New Post

Frontend (NewPost.js):

```
function NewPost() {  
  const [newPost, setNewPost] = useState("");  
  const {register, handleSubmit, formState: { errors },} = useForm();  
  const onSubmit = async (data) => {  
    const post = JSON.stringify(data);  
    try {  
      const response = await fetch("https://qshsfj-8080.csb.app/api/post", {  
        method: "post", headers: {  
          'Accept': 'application /json',  
          'Content-Type': 'application/json'  
        },  
        body: post,  
      });  
      if (response.ok) setNewPost("Post created successfully!");  
    } catch (error) {  
      console.error("Error creating data:", error);  
      setNewPost("Post created failed!");  
    }  
  };  
};
```

Add New Post

Frontend (NewPost.js):

```
return (  
  <form onSubmit={handleSubmit(onSubmit)}>  
    <div style={{ padding: 10 }}>{ " " }<br />  
    <span>Slug:</span><br />  
    <input type="text" {...register("slug", { required: true })} /><br />  
    {errors.slug && <div style={{ color: "red" }}>Slug is required</div>  
    <span>Title:</span><br />  
    <input type="text" {...register("title", { required: true })} /><br />  
    {errors.title && <div style={{ color: "red" }}>Title is required</div>  
    <span>Description:</span><br />  
    <input type="text" {...register("description", { required: true })} />  
    <br />  
    {errors.description && <div style={{color:"red"}}>Description is  
    required</div>  
    <br/><button type="submit">Add New</button>  
    <p className="text-success">{newPost}</p>  
  </div>  
</form>  
>);  
>
```

Login

Backend (index.js):

```
app.post("/api/login", bodyParser, (req, res) => {  
  const creds = {  
    username: req.body.username,  
    password: req.body.password,  
  };  
  if (creds.username === "admin" && creds.password === "123")  
  {  
    res.status(200)  
      .send({ message: "Login successful});  
  } else {  
    res.status(400).send({ message: "Login failed});  
  }  
});
```

Login

Frontend (Login.js):

```
function Login({ onLogin }) {  
  const [creds, setCreds] = useState({});  
  const [error, setError] = useState("");  
  const navigate = useNavigate();  
  const handleLogin = async () => {  
    try {  
      const response = await fetch("https://qshsfj-8080.csb.app/api/login", {  
        method: "post", headers: {  
          'Accept': 'application /json',  
          'Content-Type': 'application/json'  
        },  
        body: JSON.stringify(creds),  
      });  
      if (response.ok){  
        onLogin && onLogin({username: creds.username});  
        navigate('/stats');  
      }else setError("Invalid username or password!");  
    } catch (error) {  
      console.error("Login error:", error);  
      setError("Login failed!");  
    }  
  }  
}
```

Login

Frontend (Login.js):

```
return (
  <div style={{ padding: 10 }}> <br/>
    <span>Username:</span><br/>
    <input type="text" onChange={(e) => setCreds({...creds,
      username:e.target.value})}/><br/>
    <span>Password:</span><br/>
    <input type="password" onChange={(e) =>setCreds({...creds,
      password: e.target.value})}/><br/><br/>
    <button onClick={handleLogin}>Login</button>
    <p>{error}</p>
  </div>
);
}
```

Get Post List And Post Detail

Backend (index.js):

```
app.get("/api/posts", function (req, res) {  
  res.send(JSON.stringify(BlogPosts.BlogPosts));  
});  
  
app.get("/api/post/:slug", function (req, res) {  
  const slug = req.params.slug;  
  const post =  
    BlogPosts.BlogPosts.find((element) => element.slug === slug);  
  if (post) res.send(JSON.stringify(post));  
  else res.status(404).send("Not found");  
});
```

Get Post List

Frontend (PostLists.js):

```
export default function PostLists() {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch("https://qshsfj-8080.csb.app/api/posts");
        if (!response.ok) {
          throw new Error(`HTTP error! Status: ${response.status}`);
        }
        const result = await response.json();
        setData(result);
        setLoading(false);
      } catch (error) {
        console.error("Error fetching data:", error);
        setError("An error occurred while fetching the data.");
        setLoading(false);
      }
    };
    fetchData();
  }, []);
}
```

Get Post List

Frontend (PostLists.js):

```
return (  
  <ul>  
    {data.map((d) => (  
      <li key={d.slug}>  
        <Link to={`/${d.slug}`}>  
          <h3>{d.title}</h3>  
        </Link>  
      </li>  
    ))}  
  </ul>  
);  
}
```

Get Post Detail

Frontend (Post.js):

```
export default function Post() {
  const { slug } = useParams();
  const [post, setPost] = useState("");
  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch("https://qshsfj-8080.csb.app/api/post/"+slug);
        const result = await response.json();
        setPost(result);
      } catch (error) { console.error("Error fetching data:", error); }
    };
    fetchData();
  }, []);
  const { title, description } = post;
  return (
    <div style={{ padding: 20 }}>
      <h3>{title}</h3>
      <p>{description}</p>
    </div>);
}
```

Layout and Routes

Frontend (ProtectedRoute.js):

```
import {Navigate} from "react-router-dom";

function ProtectedRoute({user, children}) {
  if(!user) {
    return (
      <Navigate to="/login" replace/>
    );
  }

  return children;
}

export default ProtectedRoute;
```

Layout and Routes

Frontend (AppLayout.js):

```
return (  
  <>  
    <nav style={{margin: 10}}>  
      <Link to="/" style={{padding: 5}}> Home </Link>  
      <Link to="/posts" style={{padding: 5}}> Posts </Link>  
      <Link to="/about" style={{padding: 5}}> About </Link>  
      <span> | </span>  
      {user && <Link to="/stats" style={{padding: 5}}> Stats  
      </Link>}  
      {user && <Link to="/newpost" style={{padding: 5}}> New Post  
      </Link>}  
      {!user && <Link to="/login" style={{padding: 5}}> Login  
      </Link>}  
      {user && <span onClick={logOut} style={{padding: 5, cursor:  
      'pointer'}}> Logout </span>}  
    </nav>  
  </>  
);
```

Layout and Routes

Frontend (AppLayout.js):

```
<Routes>
  <Route path="/" element={<Home/>}/>
  <Route path="/posts" element={<Posts/>}>
    <Route index element={<PostLists/>}/>
    <Route path=":slug" element={<Post/>}/>
  </Route> <Route path="/about" element={<About/>}/>
  <Route path="/login" element={<Login onLogin={setUser}/>}/>
  <Route path="/stats" element={<ProtectedRoute
    user={user}><Stats/></ProtectedRoute>} />
  <Route path="/newpost" element={<ProtectedRoute
    user={user}><NewPost/></ProtectedRoute>} />
  <Route path="*" element={<NoMatch/>}/>
</Routes>
</>
);
```

Layout and Routes

Frontend (App.js):

```
function App() {  
  return (  
    <Router>  
      <AppLayout/>  
    </Router>  
  );  
}  
  
export default App;
```

Layout and Routes

Frontend (Stats.js):

- Bỏ check user khi đã dùng ProtectedRoute

```
import {Navigate} from "react-router-dom";

function Stats() {
  return (
    <div style={{ padding: 20 }}>
      <h2>Stats View</h2>
      <p>Lorem ipsum dolor sit amet, consectetur adip.</p>
    </div>
  );
}

export default Stats;
```